

CESE5040

High-Performance and AI Architectures

2026 Q4

Student Information

Name: Daniel Tyukov

Student Number: 5714699

Lab: Lab Assignment 6: The Inferior-Olive challenge

Notes

- Model: de Gruijl extended-Hodgkin-Huxley IO cell, three compartments (soma, axon, dendrite) with all-to-all gap junctions; forward-Euler, $\delta = 10 \mu\text{s}$, N cells.
- Hardware: laptop Intel Core Ultra 9 185H (22 threads) for the CPU backends; AWS Tesla T4 (16 GB, 70 W) for the GPU backend.
- Metric: mean wall-time *per simulation step*, the quantity the brief's real-time budget constrains (one step per $\delta = 10 \mu\text{s}$ of brain time). The wall time to simulate 1 s of brain time is this $\times 10^5$ steps. Each point is the best of three warm runs; JIT compilation is excluded.

Lab Exercise 6

Question 6.1: Parameter exploration

Each parameter was varied on its own (pairwise, against the defaults) and the morphology of the soma trace was measured. Table 1 summarises the effect; Figure 1 shows the two that change the dynamics qualitatively. Sub-threshold oscillations (STOs) remain near 6 Hz throughout.

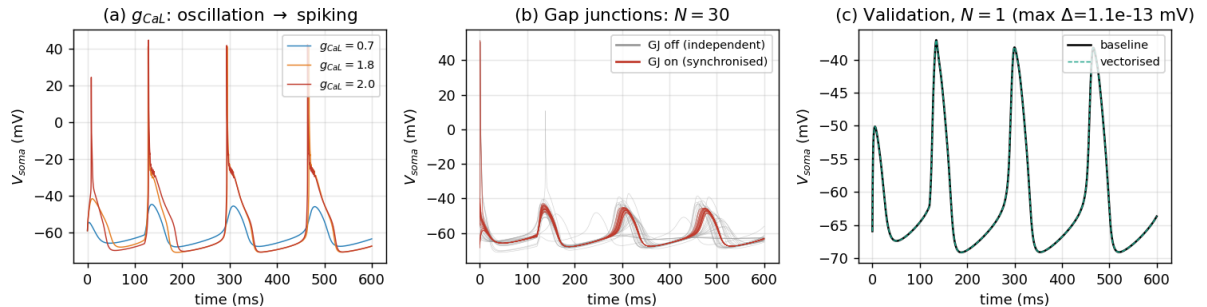


Figure 1: (a) g_{CaL} switches the cell from a sub-threshold oscillator (blue) to a Ca-spiking cell (orange/red). (b) Gap junctions bring 30 randomly-initialised cells (grey, independent) into one synchronised rhythm (red). (c) Validation: the vectorised backend reproduces the baseline trace to 1.1×10^{-13} mV at $N = 1$.

Parameter (range)	Morphological effect
sim_seconds [0.5–2.0]	No change in shape: same ~ 6 Hz STO, ~ 22 mV peak-to-peak. Only the recorded window lengthens, and runtime grows in proportion to the step count.
delta [0.01–0.05]	Single-cell trace essentially unchanged (peak-to-peak 21.6 $\rightarrow$ 21.7 mV). A coarser step means fewer steps (faster) at a small accuracy cost; its stability cost only surfaces at large N (Q6.2).
n_cells [1–30], GJ on	The isolated cell swings ± 31 mV; as the population grows the cells synchronise and each one's amplitude reduces to ~ 22 mV. Mean pairwise correlation rises 0.81 ($N=2$) $\rightarrow$ 0.97 ($N\geq 10$).
enable_gapjunctions ($N=30$)	Controls synchrony: on $\Rightarrow$ correlation 0.97 and one shared rhythm; off $\Rightarrow$ correlation 0.26, with cells on independent phases and larger individual swings (Fig. 1b).
g_CaL [0.7–2.0], $N=1$	Sets the firing mode. At 0.7 the cell only oscillates sub-threshold ($V_{\max} = -46$ mV, no spikes); at ≥ 1.8 it fires full Ca-spikes ($V_{\max} = +41$ mV, 7 Hz, 112 mV peak-to-peak), Fig. 1a.
I_pulse10ms [2–10], $N=30$	Stimulus amplitude. At 2–5 it produces a sub-threshold deflection (peak-to-peak 22 $\rightarrow$ 26 mV); at 8 it evokes a single spike; at 10 it drives an 8-spike burst and re-entrains the rhythm to ~ 2 Hz.

Table 1: Effect of each parameter on the simulator output. STO frequency stays near 6 Hz; the parameters that reshape the trace are g_CaL (firing mode), the gap junctions (synchrony) and a strong I_pulse10ms (evoked spikes).

Question 6.2: Accelerating the simulator

1 (Pen-and-paper analysis and profiling). Per time step the solver touches each of the N cells once and, for each cell, updates three compartments. A compartment update is a fixed block of ~ 40 `exp()` evaluations and arithmetic, i.e. $O(1)$ work, so the channel and compartment cost is $O(N)$ per step. The gap junction is the exception. As written it is a double loop: cell i sums $C_{\text{gap}}(V_d^i - V_d^j)$ over every other cell, which is $O(N)$ per cell and therefore

$$\text{cost per step} = \underbrace{O(N)}_{\text{compartments}} + \underbrace{O(N^2)}_{\text{gap junctions}}, \quad \text{total} = O(T N^2), \quad T = \frac{1000 \text{ sim_seconds}}{\delta}.$$

With the default $T = 10^5$ steps the quadratic term overtakes the linear one beyond a few tens of cells. Profiling the sequential code (cProfile, $N = 30$) agrees: `update_dend`, the function holding the gap-junction loop, takes 49 % of the time, `update_soma` 34 %, `update_axon` 16 % (Fig. 2a), and the measured per-step time bends away from $O(N)$ toward $O(N^2)$ as N grows (Fig. 2b). The 1.8 M Python function calls per 5000 steps are the constant-factor overhead on top.

The key observation is that the all-to-all sum collapses. For uniform linear coupling,

$$\sum_{j \neq i} C_{\text{gap}}(V_d^i - V_d^j) = C_{\text{gap}}(N V_d^i - \sum_j V_d^j),$$

so one global sum $\Sigma = \sum_j V_d^j$, formed once per step in $O(N)$, serves every cell in $O(1)$. The gap junction drops from $O(N^2)$ to $O(N)$ *exactly*. This is the “partition in space” point from the lectures: the coupling links all cells every step, but its structure is a mean field.

2 (Plan of attack). The techniques are the ones built up over Labs 2–5. Multiprocessing (Lab 2) is

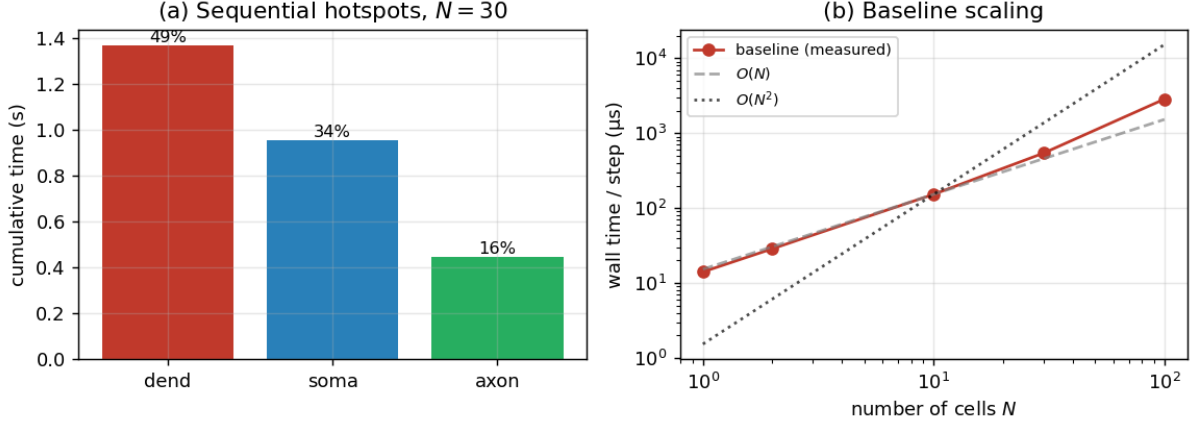


Figure 2: (a) Sequential hotspots at $N = 30$: the dendrite update (gap junctions) dominates. (b) Baseline per-step time, measured against $O(N)$ and $O(N^2)$ guides anchored at $N = 10$; the curve leaves the linear guide as the quadratic gap-junction term takes over.

left out on purpose: Lab 2 showed that for fine-grained per-timestep work the process pool’s IPC and serialisation cost exceeds the few microseconds of real work per step, so it loses to a vectorised kernel. The right family for this workload is therefore vectorise $\rightarrow$ JIT $\rightarrow$ GPU.

1. **Take the interpreter out of the hot loop.** Vectorise across cells, as in Lab 3’s `tvb_vec`: hold each state variable as a length- N array and make one timestep ~ 30 array operations instead of $3N$ Python calls (NumPy).
2. **Kill the $O(N^2)$.** Replace the gap-junction loop with the $O(N)$ mean-field sum above. Lab 3 made the same point in reverse: porting before refactoring the per-centre loop just launches more tiny kernels.
3. **Compile the step.** `Numba@njit(fastmath)` for low-overhead single-core machine code; JAX/XLA `jit+lax.scan` for a fused kernel that also runs on the GPU unchanged (both used in Labs 3–5).
4. **Offload the tail to the T4.** Run the same JAX program on the GPU for large N , the GPU path from Lab 3 (CuPy) and Lab 5 (JAX on the T4).
5. **Stay faithful.** Keep forward-Euler and the intra-cell update order identical to the baseline so the result can be validated (matches to round-off at $N = 1$, Fig. 1c; the only change at $N > 1$ is the gap junction moving from a within-step Gauss–Seidel sweep to a Jacobi update, which leaves the dynamics indistinguishable: $\max \Delta = 0.02$ mV, correlation 1.0000 at $N = 30$).

3 (Accelerated results). All four accelerated backends run the full sweep, including the 10^5 -cell case the sequential code cannot reach. Table 2 and Figure 3 give the per-step cost.

4 (Analysis: were the findings what I expected?). Mostly as expected; the exception is the stability limit noted below.

Speedups. Removing the interpreter and the $O(N^2)$ term gives large gains. At $N = 30$, Numba is $199\times$ the baseline, JAX/XLA-CPU $37\times$, plain vectorised NumPy $6.4\times$; at $N = 100$ Numba reaches $316\times$. At $N = 10^5$, which the baseline cannot reach in reasonable time, the T4 is $99\times$ faster than Numba and $134\times$ faster than NumPy.

N	1	2	10	30	100	1000	10000	100000
Baseline	14.1	28.7	152.7	543.8	2852.7	n/a	n/a	n/a
NumPy (vec.)	76.3	76.4	80.9	85.0	93.2	243.9	1369	15056
Numba (CPU)	0.09	0.19	0.93	2.73	9.04	90.7	1111	11183
JAX/XLA (CPU)	10.6	11.9	14.3	14.6	16.4	32.2	160.2	600.7
JAX/XLA (T4 GPU)	60.9	46.0	45.7	47.2	46.1	46.9	46.9	112.5

Table 2: Wall time per step (μs) versus population size. Bold marks the fastest backend at each N . The 1-second-brain-time wall is these values $\times 0.1$ s (e.g. baseline $N = 30$: 54.4 s; Numba: 0.27 s; GPU $N = 10^5$: 11.3 s).

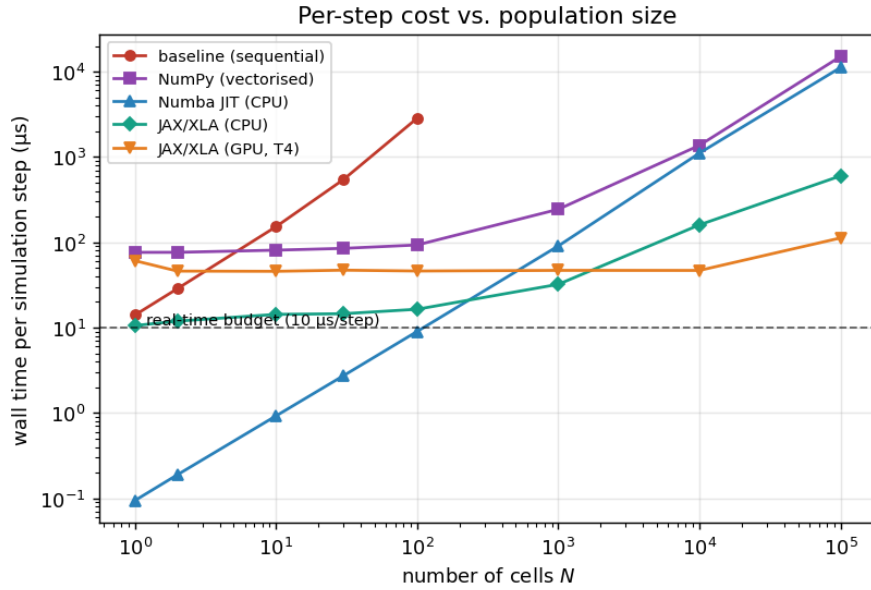


Figure 3: Per-step cost versus N (log-log). Numba is the only backend under the $10 \mu\text{s}$ real-time budget (dashed), and wins up to a few hundred cells; JAX/XLA on CPU then overtakes it; the T4 stays flat at $\sim 46 \mu\text{s}$ out to $N = 10^4$ and wins the large- N tail.

Three regimes, set by the latency–throughput trade-off from the lectures.

- **Small N ($\lesssim 10^2$): Numba wins.** With no dispatch overhead it runs at $0.09\text{--}9 \mu\text{s/step}$ and is the *only* backend inside the $10 \mu\text{s}$ real-time budget, so a single core can simulate up to ~ 100 IO cells in real time.
- **Mid N ($\sim 10^3$): JAX/XLA-CPU wins.** Once the arrays are large enough to amortise its $\sim 10 \mu\text{s}$ call overhead, XLA’s fused, SIMD-vectorised kernel beats Numba’s scalar loop.
- **Large N ($\gtrsim 3 \times 10^3$): the GPU wins.** The T4’s per-step time is *flat* at $\sim 46 \mu\text{s}$ all the way to $N = 10^4$: the work is too small to fill 2560 CUDA cores, so the step is bound by kernel-launch latency, not arithmetic. Only at $N = 10^5$ does compute finally show. This repeats Lab 3’s CuPy result, where each T4 launch cost $\sim 6\text{--}10 \mu\text{s}$ and the GPU only paid off at TVB998; here the same crossover lands near $N = 3000$. The GPU never beats the $10 \mu\text{s}$ latency, but its throughput dominates everything else offline.

This is consistent with the lecture framing that brain simulation is *not* embarrassingly parallel: the per-step global sum Σ is a synchronisation barrier across all cells, so the parallelism is in space (cells), never in time (steps stay sequential).

Stability, not compute, becomes the binding limit. The mean-field reduction makes even 10^5 cells cheap to compute, so the limiting factor at large N is numerical. With a fixed C_{gap} the coupling current grows as $C_{\text{gap}} N (V_d^i - \overline{V_d})$, and forward-Euler stability requires $\delta C_{\text{gap}} N < 2$, i.e. $N < 2/(\delta C_{\text{gap}}) = 4000$. This matches the measurement: the trajectory is stable to $N = 3000$ and diverges at $N = 4000$. Beyond that the per-step timings remain valid (the arithmetic is identical) but the dynamics are non-physical unless the coupling is normalised (C_{gap}/N) or δ is reduced. The acceleration therefore reaches a modelling limit that the sequential code was too slow to encounter.

LLM Usage Disclosure

I used Claude Code on this lab for:

- Cross-checking the $O(N^2) \rightarrow O(N)$ gap-junction derivation and the forward-Euler stability bound.
- Researching the acceleration techniques against the lecture and prior-lab material, and editing the report for concision.